

[59조] 프로젝트 기획서 v2

한글파일 양식 자동 작성 AI 어시스턴트

(가제: HwpAgent — Form-Aware Hangul Editor)

참여자 | 박재환 · 고경우 · 유승헌 · 임원재 · 정준영

버전 2.0 · 2026. 05. 03

Executive Summary

한 줄로

국가지원사업 지원서를 쓰는 학부연구생·대학원생이, 자기 연구실적·이전 제안서·CV 자료를 업로드하면, AI가 한글(.hwp) 양식의 빈칸을 자료 기반으로 자동 채워주는 대화형 웹 서비스.

기존 기획서가 한 줄짜리 메모 수준이었다면, v2는 다음을 명확히 한다.

- 페르소나를 '공공기관 근로자' → '국가지원사업에 지원하는 학부연구생·대학원생'으로 좁힘
- 킬러 기능을 '한글 편집 일반' → '양식 자동 채우기 + 데이터 취합'으로 정의
- AI 두뇌를 '의도분류기 + 실행기' → 'LangGraph 기반 적응형 상태그래프'로 전환
- MVP 포맷을 'HWP+HWPX 둘 다' → 'HWPX 우선, HWP는 V1.5'로 현실화
- 개인정보(주민번호·계좌·주소)는 LLM에 절대 보내지 않으며 AI가 작성하지도 않는다는 정책 명시
- 2주 / 5명 일정을 'Day 1-3 개인 PoC → Day 4-14 통합' 구조로 구체화

목차

1. 문제 정의 및 프로젝트 개요4
2. 사용자 페르소나6
3. Agent 설계8
4. 핵심 기능 및 사용자 흐름9
5. 기술 구현 설계13
6. UI / UX 설계18
7. 보안 · 프라이버시20
8. 성과 평가 및 KPI21
9. MVP 범위와 2주 로드맵23
10. 팀 R&R 및 협업 방식25
11. 리스크 분석 및 대응26
12. 기대 효과 및 확장 로드맵27

1. 문제 정의 및 프로젝트 개요

1.1 프로젝트 한 줄 정의

Project Statement

사용자의 기존 자료(CV, 이전 제안서, 연구실적, 프로젝트 설명서)를 입력으로 받아, 국가지원사업 .hwp 양식의 빈칸을 자동으로 채워주는 LangGraph 기반 대화형 챗봇.

1.2 서비스 한 줄 정의 (사용자 관점)

"내 자료들을 한 번에 업로드하면, 챗봇이 양식의 빈칸을 자료 기반으로 채워주고, 부족한 부분만 묻습니다. 내가 양식 한 번 더 들여다볼 일 없이 검토만 하면 끝."

1.3 서비스 선정 배경

국가지원사업(BK21, 학부생연구지원, 한국연구재단 사업, 창업지원사업 등)은 공통적으로 다음 특성을 가진다.

- 지원 양식이 .hwp / .hwp로 배포되며, 양식의 표·문항·서식이 매우 엄격함
- 지원자가 같은 정보(연구실적, 자기소개, 이전 프로젝트)를 매번 양식에 맞춰 다시 작성해야 함
- 사업마다 양식 구조가 달라 단순 복붙이 통하지 않고, 표 안의 셀, 글자 수 제한 등 미시 조정이 많음
- 기존 한컴오피스의 AI 어시스턴트는 단발성 명령에 강하지만, '여러 자료를 종합해서 양식 전체를 채우는' 워크플로우에는 약함

결과적으로 지원자는 양식 한 건당 평균 5~10시간을 "양식과 싸우는" 데에 쓴다. 이 시간을 자료 정리·내용 작성·검토로 옮기는 것이 본 서비스의 목표.

1.4 해결하려는 문제

Core Problem

내 정보는 '여러 파일에 흩어져 있고', 양식은 '한 곳에 모이도록 강제한다.' 사람이 양식의 빈칸 하나하나에 어떤 정보가 들어가야 하는지 머릿속에서 매핑하고, 길이를 맞추고, 표 셀을 찾아 채우는 인지 비용이 본질적인 병목이다.

이를 다음 세 부분 문제로 분해한다.

1. [양식 이해] 양식의 어느 부분이 '채워야 할 빈칸'이며, 어떤 종류의 정보를 기대하는가?
2. [자료 매핑] 사용자의 자료들 중 어느 부분이 그 빈칸과 연결되는가?
3. [정합성] 채운 결과가 양식의 형식·문체·맥락과 어울리는가?

1.5 대상 사용자

[1차 타겟] 국가지원사업 지원서를 제출해야 하는 학부연구생 · 대학원생

[2차 타겟] 이공계 연구실, 연구재단 사업에 자주 응모하는 신진 연구자

(*"공공기관 근로자" 페르소나는 데이터 보안·망분리·온프레미스 요구로 인해 V1 범위에서 명시적으로 제외하고, V2 이후 확장 후보로 둔다.*)

1.6 핵심 가치

관점	핵심 가치
시간	양식 1건당 5~10시간 → 1~2시간으로 단축 (목표치)
인지 부하	"양식 어디 채우지"에서 "내용을 검토하고 다듬자"로 작업의 본질이 이동
정확성	자료 기반 작성이라 사실 오류·부풀리기 감소, 근거 표기 가능
재사용성	같은 자료를 여러 양식에 반복 활용. 자료를 '내 지식 베이스'로 시동시킬 수 있는 발판

2. 사용자 페르소나

2.1 1차 페르소나 — 김연구 (24세, 학부 4학년)

배경	이공계 학부 4학년, 학부연구생. 1년차 학부연구지원사업에 처음 지원. 한글 사용 빈도는 높지 않으나 학교에서 .hwp 양식을 받아본 적 있음. 영문 논문은 LaTeX, 발표는 PPT, 한글은 거의 처음.
Pain Point	(1) 양식의 표가 어디까지가 의무 항목인지 모름. (2) 어떤 정보가 어디에 들어가야 하는지 가이드가 부족. (3) 글자 수 제한·페이지 수 제한에서 매번 시간 소비. (4) 지도교수에게 무엇을 물어봐야 할지조차 불명확.
기대 행동	내 CV·연구계획·이전 보고서 파일을 한 번에 업로드 → AI가 "이 양식은 이런 구조이고, 자료에서 이런 정보가 매칭됩니다. 다만 ○○ 부분은 추가 정보가 필요해요"라고 안내 → 대화하며 단계적으로 채움.
성공의 정의	"AI가 만든 초안을 80% 그대로 쓰고, 20%만 다듬어서 제출."

2.2 2차 페르소나 — 박대학 (28세, 석사 2학기)

배경	이공계 석사 2학기. 매 학기 BK21·연구장학금·학회 발표 지원 등에 응모. 한글 양식에 익숙하나 매번 같은 자료를 다시 정리하는 것이 부담.
Pain Point	(1) 같은 자기소개·연구실적을 양식별로 길이·강조점 다르게 써야 함. (2) 표 안의 "게재 논문" 칸을 매번 새로 정리. (3) 기관마다 요구하는 추진계획·예산 항목이 다름.
기대 행동	'내가 갖고 있는 정보'는 그대로 두고, 양식만 바뀌었을 때 AI가 양식의 톤·길이에 맞춰 자동 변환해주기를 원함.
성공의 정의	"내 자료 1세트로 양식 3개를 30분 안에 채울 수 있다."

2.3 비-페르소나 (명시적 제외)

- 공공기관 근로자: 망분리·온프레미스·보안 검토가 필요해 학기 일정에 부적합 (V2 이후)
- 법무·행정 문서 작성자: 한글 양식이지만 법률 검토 책임이 따라옴 (범위 외)
- 개인정보가 핵심인 양식(예: 보건 의료 신청서): 7장 보안 정책상 핸들링 위험 (범위 외)

3. Agent 설계

3.1 역할 정의

Agent는 "꼼꼼한 행정 조교"의 메타포를 가진다. 사용자보다 양식의 구조를 먼저 파악하고, 자료에서 필요한 정보를 찾아오고, 부족하면 질문한다. 단, 사용자의 최종 결정권을 침범하지 않는다.

v1 Agent와의 차이

v1: "팀원의 업무 부담을 줄이려고 스스로 모든 일을 하려는 팀장" — 직관적이지만 '팀장'은 자율 결정권을 함의해 위험.v2: "양식과 자료를 모두 본 적 있는 행정 조교" — 항상 사용자의 승인 안에서만 움직이는 보조자 메타포로 명확화.

3.2 자율성 범위

범위	자율 행동	사용자 승인 필요
양식 분석	빈칸 식별, 항목 분류, 글자 수 추정	(자율)
자료 해석	자료별 요약, 항목과의 매핑 후보 도출	(자율)
초안 작성	매핑된 정보로 초안 생성	매 항목 단위로 미리보기 제시
양식 수정	(없음)	초안 → 사용자 승인 → 양식에 반영
개인정보	(없음)	AI는 작성하지 않음. [사용자 직접 입력] 마킹.

3.3 톤앤매너

- 공식적이고 간결한 한국어 (지원서 톤)
- 불확실하면 추정하지 않고 질문 ("이 부분에 대한 추가 정보가 필요합니다")
- 자료 기반 작성 시 가능하면 "○○ 자료를 참고했습니다"라고 출처 표기

- 사용자가 거절·수정 요청할 때 변명·반복 사과 없이 곧바로 재작성

4. 핵심 기능 및 사용자 흐름

4.1 V1 핵심 기능 (MVP)

ID	기능	설명
F1	양식 자동 채우기	업로드된 .hwp 파일의 빈칸·표를 LLM이 식별 → 자료 기반 초안 생성 → 단계별 미리보기 → 사용자 승인 후 양식에 반영
F2	표 편집·데이터 채우기	양식 내 표(연구실적, 게재논문, 추진일정 등)의 행/열 구조를 인식하여 자료에서 데이터를 매칭 채움
F3	서식·구조 일괄 적용	스타일·머리말·페이지번호 등을 양식 원본 규칙대로 유지. 사용자가 "○○ 양식대로" 명령 시 동기화
F4	대화형 정보 보충	정보가 부족하면 즉시 질문 노드로 분기 → 사용자 답변을 받아 작성 재개
F5	후속 명령 라우팅	양식 채우기 중 "3번 다시", "전체 톤 더 공식적으로" 등 후속 명령을 분류해 적절한 서브그래프 호출
F6	개인정보 마스킹	자료 업로드 시 정규식 + Presidio로 PII 자동 식별 → LLM 입력 전 마스킹 → 양식의 PII 필드는 [사용자 직접 입력]으로 비워둠

4.2 명시적 비-기능 (Out of Scope for V1)

- .hwp 바이너리 포맷 직접 핸들링 (V1.5)
- 글자 수·표 행 수 등 양식 제약 자동 검증·재작성 (V2)
- 되돌리기/되살리기 — 사용자는 "다시 써" 명령으로 재생성 (단순화)
- 사용자 계정·장기 지식 베이스 — 모든 자료는 세션 단위 (보안 단순화)
- 실시간 협업, 코멘트, 트래킹

4.3 대표 사용자 시나리오

시나리오 A — "내 자료로 한 번에 채우기" (해피 패스)

4. 김연구가 학부연구지원 양식.hwp와 본인 CV.pdf, 이전 보고서.docx, 연구계획.txt를 업로드
5. AI: "양식 분석 중... 23개 항목 발견. 자료에서 18개 항목 매칭됩니다. 5개 항목은 추가 정보가 필요해요. 시작할까요?"
6. 승인 → 항목별 미리보기 → 김연구가 항목 단위로 "좋아요 / 다시 써주세요 / 이렇게 바꿔주세요"
7. 미매칭 5개 항목에 대해 AI가 차례로 질문 → 답변으로 채움
8. 최종 .hwp 다운로드. 단, 신청자 인적사항(이름·학번·연락처)은 [사용자 직접 입력] 마킹

시나리오 B — "이 부분만 다시" (후속 명령)

9. 초안 검토 중 김연구가 "3번 항목, 연구의 필요성을 더 학술적인 톤으로 다시 써주세요. 1단락으로요."
10. Classifier 노드: [항목별 재작성 + 톤 조정 + 길이 제약] 분류
11. Generator 서브그래프 호출 → 새 초안 → 미리보기 → 승인 후 반영
12. 이때 다른 항목들의 상태(이미 승인된 1, 2번 등)는 건드리지 않음

시나리오 C — "자료가 부족해요" (정보 보충 흐름)

13. 양식의 "본인 연구 강점" 항목에 매칭되는 자료가 없음
14. AI: "이 항목은 자료에서 직접적인 근거를 찾기 어려웠어요. 본인이 생각하는 강점 2~3가지를 알려주실 수 있나요? 예: 도구 숙련도, 도메인 지식, 협업 경험."
15. 김연구가 한 줄씩 답변 → AI가 양식 톤에 맞게 정리해 다시 미리보기

시나리오 D — "자료 추가" (런타임 자료 보강)

16. 작성 중 김연구가 "아 잠깐, 이 발표자료도 참고해주세요" + 파일 업로드
17. Classifier 노드: [자료 추가] 분류
18. PII 마스킹 → 자료 요약 노드 → 그래프 상태의 "자료 컨텍스트"에 추가 → "3, 7, 11번 항목에 새 정보가 반영될 수 있어요. 다시 작성할까요?"

시나리오 E — "개인정보 필드" (안전 분기)

19. 양식의 "주민등록번호", "계좌번호" 셀을 식별
20. AI는 채우지 않음. "이 칸은 개인정보라 직접 입력해주세요" 안내 + 셀에 [본인 작성] 플레이스홀더만 삽입

4.4 사용자 워크플로우 (UI 관점)

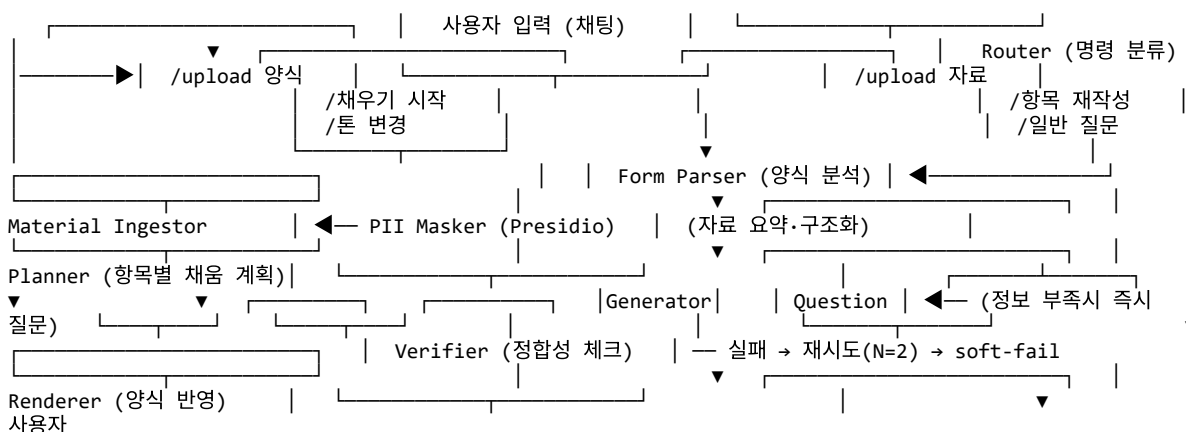
Streamlit 채팅 UI 1개 + 사이드바의 자료 업로드 영역. 흐름은 단일 채팅창에서 자연스럽게 진행.

순서:

21. [Step 1] 양식 .hwpX 업로드
22. [Step 2] 참고 자료 업로드 (PDF / docx / txt / hwpX)
23. [Step 3] AI가 양식 분석 결과를 채팅에 출력 (양식 구조 시각화)
24. [Step 4] 항목별 미리보기 → 사용자 승인 / 수정 / 질문 응답
25. [Step 5] 개인정보 필드 안내 → 사용자가 직접 입력
26. [Step 6] 최종 .hwpX 다운로드

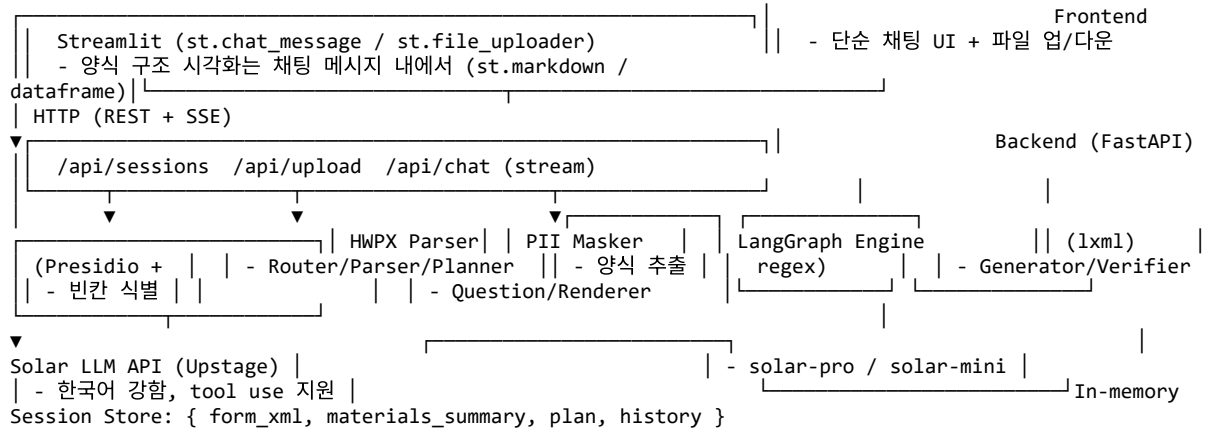
4.5 시스템 워크플로우 (LangGraph)

그래프는 메인 그래프(엔트리)와 6개 서브그래프로 구성된다. 자세한 노드 설계는 5.3절 참조.



5. 기술 구현 설계

5.1 시스템 아키텍처



5.2 기술 스택

계층	기술	선정 근거
프론트엔드	Streamlit	학기 일정 내 빠른 채팅 UI 구현. st.chat_message + 파일 업로더로 V1 충분.
백엔드 API	FastAPI	LangGraph 비동기 실행, SSE 스트리밍, Streamlit과 분리해 추후 다른 클라이언트 확장 여지.
AI 오케스트레이션	LangGraph 0.2+	상태그래프 기반 워크플로우. 분기·재시도·인터럽트(인간 개입) 표현이 LangChain Agent보다 명시적.
LLM	Upstage Solar Pro / Solar Mini	한국어 공식 문체 품질 최상. tool-use(structured output) 지원. 학기 프로젝트 비용 관리에 유리.
HWPX 처리	lxml + zipfile + python-hwp (필요 시 자체 구현)	HWPX는 ZIP+XML 구조라 docx와 동일 패턴으로 직접 파싱·편집·리패킹 가능.
자료 파싱	pypdf, python-docx, pdfplumber	PDF-docx-txt에서 일반 텍스트 추출. 표 인식 보강으로 pdfplumber.

PII 마스킹	정규식 + Microsoft Presidio (한국어 커스텀)	주민/계좌/카드/전화/이메일은 정규식 1차, 이름.주소.소속은 Presidio NER. 7장 참조.
세션 저장소	In-memory dict (Redis로 확장 가능)	개인정보 보호 정책상 디스크 영속 X. 세션 종료 시 GC.
배포·실행	로컬 / Docker compose	발표 데모는 로컬 또는 단일 컨테이너. 클라우드 배포는 V2.

5.3 LangGraph 그래프 설계

그래프 상태(GraphState)는 다음 필드를 갖는다.

FormDoc · MaterialBundle · ItemPlan[] · ConversationHistory · PendingQuestion · Errors[]

주요 노드:

노드	입력	출력
Router	UserMessage	Intent $\in$ { upload_form, upload_material, start_fill, rewrite_item, change_tone, add_material, general_qa }
FormParser	.hwp XML	FormDoc { sections[], items[], tables[], placeholders[] }
MaterialIngestor	자료 파일들	MaterialBundle { docs[] (PII-masked, summarized) }
Planner	FormDoc + MaterialBundle	ItemPlan[] = [{item_id, source_evidence[], confidence, needs_question}]
Generator	ItemPlan + 컨텍스트	DraftItem { text, citations[] }
Question	ItemPlan(needs_question=true)	PendingQuestion → 사용자 응답 후 Generator로 복귀

Verifier	DraftItem	ok / retry(이유) / soft_fail. 최대 N=2 재시도 후 soft_fail.
Renderer	승인된 DraftItem	FormDoc XML에 in-place 삽입 → 사용자에게 미리보기

5.3.1 후속 명령 라우팅 (적응형)

Router가 명령 종류를 분류하여 적절한 서브그래프를 호출한다.

- **rewrite_item / change_tone** → Planner는 건너뛰고 해당 항목만 Generator + Verifier
- **add_material** → MaterialIngestor + 영향받는 항목 재계획
- **start_fill** → 전체 그래프 (FormParser → MaterialIngestor → Planner → ...)
- **general_qa** → 양식 컨텍스트만 참조하고 채우기 액션 없이 응답

5.4 자료 처리 및 PII 마스킹 파이프라인

27. 파일 수신 (PDF / docx / hwp / txt)

28. 파서로 일반 텍스트 추출

29. 정규식으로 1차 마스킹: 주민번호 13자리, 계좌, 카드, 전화, 이메일, 금액

30. Presidio (커스텀 한국어 recognizer)로 2차 마스킹: 이름, 주소, 소속, 학번

31. 마스킹된 텍스트만 LLM에 전달. 원문은 메모리에 보관하되 LLM 호출에는 사용 안 함

32. 양식의 PII 필드도 동일 정책으로 식별 → AI는 작성하지 않음 → [사용자 직접 입력]
플레이스홀더 삽입

5.5 양식 파싱 및 빈칸 식별

v1은 LLM 전체 스캔 + 추론 방식. 다음 단계로 동작.

33. HWPX 언팩 → contents.xml의 <hp:p> 단락, <hp:tbl> 표 추출

34. 표 셀과 단락을 일정 토큰 크기로 청킹

35. 결과 통합 → FormDoc의 items[]

표(예: 연구실적 표)는 헤더 행을 인식하고, 데이터 행이 비어있으면 "행 단위 채우기" 항목으로 등록한다.

5.6 프롬프트 설계 전략

프롬프트는 노드별로 분리하며, 각 노드는 명확한 단일 책임만 갖는다.

5.6.1 Planner 프롬프트 구조

System Prompt (요약)

당신은 국가지원사업 지원서 양식의 빈칸을 사용자 자료로 채우는 행정 조교입니다. FormDoc과 MaterialBundle을 보고, 각 item에 대해 (1) 어떤 자료가 매칭되는가, (2) 매칭이 충분한가, (3) 사용자에게 추가 질문이 필요한가를 JSON으로 출력하세요. 추측하지 말고 자료에 명시된 내용만 evidence로 인정합니다.

5.6.2 Generator 프롬프트 구조

System Prompt (요약)

당신은 다음 항목의 초안을 작성합니다. [항목 정보] [자료 evidence] [길이 가이드] [톤 가이드(공식적/학술적)]를 받습니다. evidence에 없는 내용은 작성하지 마세요. 추정해야 할 경우 [확인 필요] 마킹을 남깁니다. 출력 형식: { text, citations[] }.

5.6.3 Verifier 프롬프트 구조

System Prompt (요약)

다음 초안이 (1) 항목 라벨에 답하고 있는지, (2) evidence 외 내용을 환각하지 않았는지, (3) 톤이 공식적인지 평가하고 ok / retry(reason)로 응답하세요.

5.7 메모리·상태 관리

- 그래프 상태는 세션당 1개. 메모리에서만 유지.
- 대화 히스토리는 최대 10턴 슬라이딩 윈도우 (LangGraph state의 messages 필드)
- FormDoc-MaterialBundle은 세션 동안 풀버전 유지 (양식·자료는 처음부터 끝까지 필요)

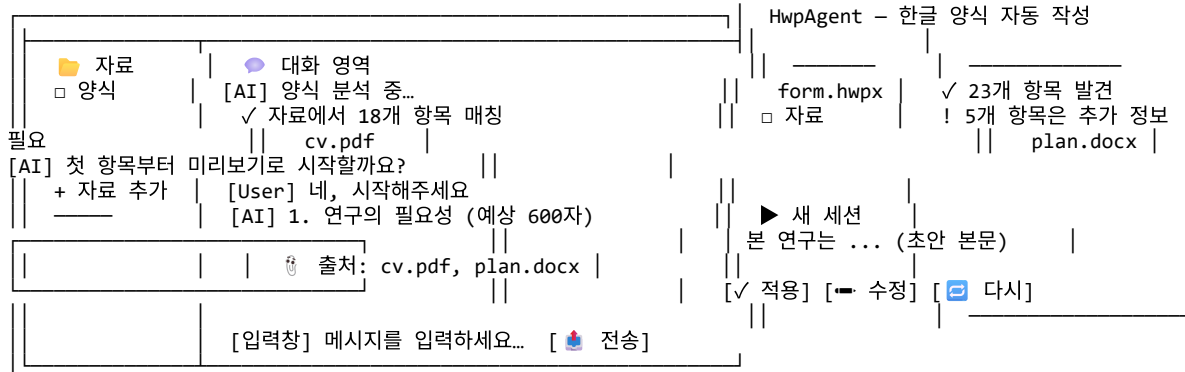
- 승인된 DraftItem들은 ItemPlan에 누적 저장 → 후속 "3번 재작성" 시 다른 항목 영향 없음

5.8 에러 처리 및 폴백

에러 지점	대응
Solar API 5xx / 타임아웃	지수 백오프 재시도 3회 → 실패 시 "잠시 후 다시 시도해주세요" 소프트 실패. 그래프 상태 보존.
HWPX 파싱 실패	파일이 손상/암호화됐을 가능성. 사용자에게 "이 양식 파일을 열 수 없어요. 파일 형식을 확인해주세요" 안내.
Verifier 재시도 한계 초과	해당 항목은 [확인 필요] 마킹 + 사용자에게 "제가 이 항목을 자신 있게 채우지 못했어요. 직접 작성해주실 수 있나요?"
PII 마스킹 누락 의심	Generator 출력에 13자리 숫자/계좌 패턴 발견 시 자동 차단 + 재생성. 재시도 후에도 패턴 유지 시 [확인 필요].
Router 분류 실패 (low confidence)	"이 명령을 어떻게 이해하면 될까요? (a) 항목 재작성 (b) 새 자료 추가 (c) 일반 질문" 사용자에게 disambiguation.

6. UI / UX 설계

6.1 화면 구조 (Streamlit)



6.2 사용자 명령 → 응답 흐름

모든 상호작용은 단일 채팅 스레드에서 일어난다. 사이드바는 현재 업로드된 자료 목록만 보여주는 정적 영역.

AI 응답은 최소 두 가지 시각 요소를 함께 사용한다:

- [채팅 메시지] 자연어 안내 + 항목 미리보기 (markdown 렌더링)
- [액션 버튼] [✓ 적용], [✎ 수정], [↺ 다시] (Streamlit st.button 인라인)

6.3 미리보기 표현 — 양식 구조 시각화

"단계별 미리보기"의 구체적 표현 수단을 다음과 같이 정한다.

6.3.1 일반 단락 항목

채팅 메시지에 직접 인용 블록 형태로 표시:

3. 연구의 필요성 (예상 600자, 자료 기반)

(초안) 본 연구는 ... (생성된 텍스트) 출처: cv.pdf §2, plan.docx §1

6.3.2 표 항목

st.dataframe 또는 markdown 표로 시각화. 빈 셀은 [△ 추가정보 필요], PII 셀은 [👉 직접입력].

6.3.3 양식 전체 구조

"양식 분석 중..." 응답 시점에 expander/트리 형태로:

양식 구조 요약

▼ 1. 신청자 정보 (5개 항목, 4개는 PII → 직접 입력) ▼ 2. 연구 개요 (3개 항목, 모두 자료 매칭 ✓) ▼ 3. 추진 계획 (1개 표 + 2개 항목, 표 데이터 추가 필요) ▼ 4. 예산 (1개 표, 항목별 단가 정보 필요)...

6.4 후속 명령 처리 UX

후속 명령은 두 가지 입력 경로:

36. 자유 채팅 입력 — "3번 다시", "전체 톤 더 공식적으로". Router가 분류.

37. 미리보기 옆 액션 버튼 — [ 다시], [ 수정 (사용자 지시)]. 명시적이고 빠름.

두 경로 모두 같은 그래프 라우팅 결과로 수렴한다. 버튼은 Router를 우회하지 않고 명시적 intent를 채워서 그래프에 진입한다.

7. 보안 · 프라이버시

핵심 원칙 (절대 위반 금지)

(1) 개인정보는 LLM에 절대 보내지 않는다.(2) AI는 양식의 PII 필드를 작성하지 않는다.(3) 사용자 자료·대화·양식은 디스크에 영속 저장하지 않는다 (메모리 온리).(4) 세션 종료 시 즉시 GC.

7.1 PII 정의

다음은 PII로 분류하고 마스킹·작성 차단 대상으로 둔다.

- 주민등록번호, 외국인등록번호, 여권번호
- 계좌번호, 카드번호, 금액 정보(개인 자산 명시 시)
- 전화번호, 이메일 주소
- 실명, 가족 관계명, 주소
- 학번, 사번, 학생증/사원증 번호
- 주민번호 형식의 임의 13자리 숫자열

7.2 마스킹 파이프라인

38. 1차: 정규식 (주민번호 13자리, 계좌 패턴, 전화, 이메일, 카드)

39. 2차: Presidio NER + 한국어 커스텀 recognizer (이름, 주소, 소속, 학번)

40. 3차: Generator 출력 사후 검사 — PII 패턴 누출 감지 시 차단·재생성

7.3 양식 PII 필드 처리

FormParser가 항목 라벨을 분석하여 PII 필드 후보를 식별한다.

- "성명", "주민등록번호", "연락처", "주소", "계좌", "학번" 등 키워드 매칭
- 매칭된 필드는 ItemPlan에서 type=PII로 분류
- Generator는 PII 항목을 스킵, Renderer는 [본인 직접 입력] 플레이스홀더 삽입

- 최종 다운로드 직전 사용자에게 "○개 PII 항목은 직접 작성이 필요해요" 안내

7.4 LLM 프로바이더 정책

- Solar API 호출 시 "학습 비활용" 옵션 명시 (Upstage 정책 확인 후)
- 로깅에서 사용자 입력·출력 텍스트 제외 (메타데이터만 기록)

7.5 발표·시연 시 주의

- 실제 본인 자료 사용 금지. 가공된 데모용 자료만 사용.
- 스크린샷·발표자료에 마스킹된 텍스트도 그대로 노출하지 않음.

8. 성과 평가 및 KPI

2주 / 5명 / 학기 프로젝트 특성을 고려해 "실사용자 적중률"보다는 "프로젝트 완성도"에 가중치를 두는 KPI 구성을 채택한다.

8.1 정량 KPI

ID	KPI	측정 방법	목표치
K1	양식 빈칸 식별 정확도	3~5개 양식의 빈칸을 사람이 정답화 → AI 식별과 비교 (P/R/F1)	F1 $\geq$ 0.85
K2	Router 의도 분류 정확도	50개 명령 테스트셋 (재작성/톤변경/자료추가/QA 등)	Accuracy $\geq$ 0.90
K3	양식 채움률 (자율)	"자료 + 양식" 페어에서 AI가 추가 질문 없이 자율적으로 채운 항목 비율	$\geq$ 60%
K4	Verifier 재시도율	Generator 출력의 Verifier 1차 통과 비율 (안정성 지표)	$\geq$ 75%
K5	PII 누출 0건	PII 포함 데모 자료 20개 입력 → LLM 입력·출력 로그 검사	0건 (필수)
K6	End-to-End 응답 시간	"채우기 시작" 명령 → 첫 항목 미리보기까지	$\leq$ 30초 (p50)

8.2 정성 KPI (프로젝트 완성도)

- 3~5개 실제 양식에 대한 End-to-End 데모 시나리오 5개 모두 통과
- LangGraph 그래프 다이어그램이 코드와 1:1 일치하며, 노드별 책임이 README에 문서화됨
- PII 마스킹 파이프라인이 단위 테스트 50개 이상 통과
- 팀원 5명이 각자의 PoC를 발표하고 통합 결정 근거를 회고록으로 남김
- 발표 시 5분 내 라이브 데모 가능

8.3 평가셋 구축 계획

41. 실제 공개된 양식 3~5개 수집 (학부생연구지원, BK21, 학회 발표 지원 등 비식별 가능 양식)
42. 각 양식에 대한 "가상 지원자" 자료셋 3종 구성 (CV·연구계획·이전보고서 mockup)
43. 정답 항목 추출(K1)·정답 분류 라벨(K2)을 사람이 작성
44. 회귀 테스트 자동화 (pytest + LangGraph 단위 테스트 + E2E)

9. MVP 범위와 2주 로드맵

9.1 V1 / V1.5 / V2 범위

범위	V1 (MVP, 2주)	V1.5 (학기 후반)	V2 (확장)
포맷	HWPX	.hwp 추가 (libreoffice 변환)	.hwp 네이티브, docx 일부
기능	F1~F6 (4.1절)	글자 수·표 행 제약 자동 검증	장기 KB / RAG, 사용자 계정
배포	로컬 / Docker	단일 서버 클라우드	멀티테넌트, 공공기관용 온프레미스
UI	Streamlit 단일 채팅	PDF 미리보기 추가	React 전용 UI / 한컴 플러그인

9.2 2주 일정 (Day 1~3 개인 PoC + Day 4~14 통합)

Phase 1 — Day 1~3 : 개인 탐색·PoC

- **목표:** 5명이 각자 "양식 업로드 → 자료 업로드 → 채워서 다운로드" 미니 파이프라인을 끝까지 한 번씩 만들어본다.
- 개인별 자유 — Streamlit 직접 / FastAPI / LangGraph 전부 한 사람이 / 일부만
- Day 3 끝에 5개 PoC 데모 + "무엇을 어떻게 했는지" 공유 회의
- 산출물: 각자의 GitHub 브랜치 + 짧은 회고 메모

Phase 2 — Day 4 : 통합 설계 결정

- 5개 PoC 비교 → 통합 아키텍처 결정 (한 명의 베이스 + 다른 사람 강점 흡수, 또는 from-scratch 통합)
- R&R 확정 (10장)
- 산출물: 통합 아키텍처 다이어그램 + 모듈 인터페이스 합의

Phase 3 — Day 5~10 : 통합 구현

- Day 5~6: HWPX Parser + PII Masker 모듈 안정화

- Day 5~7: LangGraph 노드 구현 (Router → FormParser → MaterialIngestor → Planner)
- Day 7~9: Generator + Verifier + Question + Renderer
- Day 8~10: Streamlit + FastAPI 통합 + SSE 스트리밍
- Day 9~10: 평가셋 구축 (양식 3~5개, 가상 자료 3세트)

Phase 4 — Day 11~13 : 통합 테스트·튜닝

- E2E 시나리오 5개 회귀 테스트
- 프롬프트 튜닝 (특히 Planner / Verifier)
- PII 마스킹 단위 테스트 50개 이상
- KPI 측정 및 보완

Phase 4 — Day 14 : 발표 준비

- 5분 라이브 데모 시나리오 리허설 ×3
- 발표자료 + 시연 영상 백업
- 회고 / 한계 / 향후 확장 정리

9.3 마일스톤

Day	마일스톤
D3	개인 PoC 5개 데모 완료
D4	통합 아키텍처 합의
D7	"Hello World" — 빈 양식 업로드 → AI가 항목 1개 채운 결과 다운로드
D10	E2E 시나리오 A(해피 패스) 통과
D12	E2E 시나리오 5개 모두 통과 + KPI 측정 데이터
D14	발표·시연

10. 팀 R&R 및 협업 방식

10.1 협업 방식

"각자 전체 파이프라인 만들기 → 공유 → 통합" (Day 1-3 개인 PoC → Day 4 통합 결정).

개인 PoC 단계에서는 학습 효과 극대화를 노리고 모든 멤버가

LangGraph·HWPX·Streamlit·LLM 호출을 모두 한 번씩 만져본다.

Day 4 이후 R&R은 PoC에서 가장 잘 해낸 영역으로 자연스럽게 분배한다 (예시는 아래).

10.2 R&R 분배 후보 (Day 4에 확정)

역할	주요 책임
HWPX & 양식 파싱	HWPX 언팩/리패킹, FormParser 노드, 항목 식별 휴리스틱 + LLM 보강
LangGraph 그래프	그래프 정의, 상태 관리, Router/Planner/Verifier 노드, 재시도·인터럽트 로직
Generator & 프롬프트	Solar 프롬프트 설계, Generator/Question 노드, 평가셋 기반 프롬프트 튜닝
프론트·백엔드 통합	Streamlit UI, FastAPI 엔드포인트, SSE 스트리밍, 파일 업/다운
PII·테스트·문서화	Presidio 한국어 커스텀, PII 단위 테스트, 평가셋 구축, KPI 측정 자동화, README

10.3 협업 도구

- 코드: GitHub (단일 레포, feature 브랜치 + PR 리뷰)
- 커뮤니케이션: Slack / Discord 채널 1개
- 일정: Notion 또는 GitHub Project
- 회의: 매일 10분 스탠드업 (어제 / 오늘 / 막힌 점)
- 페어/모브: 통합 단계(Day 4~7) 핵심 모듈은 페어로

11. 리스크 분석 및 대응

ID	리스크	영향	확률	대응
R1	HWPX 라이브러리 부재·미성숙	높음	중간	Day 1~2 PoC에서 HWPX 직접 파싱·리패킹 검증. 막히면 "빈 양식 + AI가 해당 셀 텍스트만 교체"로 단순화.
R2	Solar API 한국어 톤·구조 안정성	중간	중간	Verifier로 1차 차단 + 프롬프트 few-shot 강화. 비상 시 OpenAI/Claude 폴백 어댑터 구조 미리 준비.
R3	PII 마스킹 누락 → 정보 유출	매우 높음	낮음~중 간	정규식+Presidio 이중 + Generator 사후 검사 3중. PII 단위 테스트 50개 이상 + 의심 시 차단 우선.
R4	Streamlit의 채팅 UI 한계	중간	중간	미리보기는 마크다운 + dataframe로 충분. PDF 미리보기 등은 V1.5로 미룸.
R5	LangGraph 학습 곡선	중간	중간	Day 1~3 개인 PoC 단계에서 모두가 미니 그래프 1개씩 작성. Day 4 통합 시 가장 잘된 형태 채택.
R6	일정 슬립 (2주 빠듯함)	높음	중간	D7 "Hello World", D10 시나리오 A 통과의 게이트로 진척 측정. 미달 시 시나리오 수 축소.
R7	LLM 환각 (없는 사실 작성)	높음	높음	Generator 프롬프트에 "evidence-only" 강제 + Verifier가 evidence 외 내용 탐지 + citations 노출.

12. 기대 효과 및 확장 로드맵

12.1 기대 효과

- **[정량]** 양식 1건 작성 시간 5~10시간 → 1~2시간 (목표). 같은 자료로 양식 N개 작성 시 비용 함수가 거의 상수에 가까워짐.
- **[정성]** 지원자가 "양식과 싸우는" 인지 부담을 "내용 검토에 집중"으로 옮긴다. 자료 기반 작성으로 사실 정확성·근거 표기 강화.
- **[학습]** 팀 5명이 LangGraph 상태그래프, HWPX XML 처리, PII 마스킹, LLM 평가셋 구축이라는 네 가지 실무 스킬을 동시에 경험.

12.2 확장 로드맵

- **V1.5** — .hwp 포맷 지원 (libreoffice 변환), 양식 제약 검증(글자 수·표 행), PDF 미리보기
- **V2** — 장기 지식 베이스(RAG), 사용자 계정, 양식 템플릿 라이브러리, 멀티 양식 일괄 작성
- **V3** — 공공기관용 온프레미스 배포, 한컴오피스 플러그인, 결재 라인 연동, 워드(.docx) 양식 확장

12.3 마치며

v1 기획서는 "한글파일을 AI로 편집하면 좋겠다"는 아이디어 단계였다. v2는 그 아이디어를 (1) 누가, (2) 어떤 양식을, (3) 어떤 자료로, (4) 어떤 그래프로, (5) 어떤 안전장치 위에서, (6) 2주 안에 어떻게 만들 것인지로 옮겼다.

핵심 결정 다섯 가지를 다시 못 박는다.

- 45. 페르소나: 국가지원사업에 지원하는 학부연구생·대학원생
- 46. 킬러 기능: "내 자료 → 양식 자동 채우기" (단순 한글 편집 챗봇이 아님)
- 47. 아키텍처: LangGraph 기반 적응형 상태그래프, Solar LLM, Streamlit + FastAPI
- 48. 안전: 개인정보 LLM 미전송 + AI 미작성 + 메모리 온리
- 49. 일정: Day 1~3 개인 PoC → Day 4 통합 결정 → Day 14 발표